

Serial Binary Multiplier

Abstract—This project implements a compact 8×8-bit serial binary multiplier at the transistor level in 45nm CMOS with $V_{DD} = 1.2V$, targeting low-power RISC applications. In a serial binary multiplier operating over multiple clock cycles, the critical path is determined by the maximum delay of either the adder-multiplexer path or the shift operation. Since the adder delay is significantly larger than the shift delay, and the multiplexer delay is negligible compared to the adder, the timing constraint is effectively set by the adder alone. The adder used in this design exhibits a propagation delay of 0.72 ns; thus, the maximum allowable clock cycle in simulation is approximately 1 ns, accounting for an additional 0.2 ns due to rise/fall times.

Dynamic power is estimated by:

$$P_{dynamic} = \alpha C_{total} V_{DD}^2 f$$

where α is the activity factor, C_{total} the total switched capacitance, $V_{DD} = 1.2V$, and $f = 50$ MHz.

Assuming:

- Flip-flop load 10 fF
- Adder bit load 10–15 fF
- MUX load 5–10 fF

and an architecture consisting of 8 flip-flops for Q, 8 flip-flops for A, one 8-bit adder, and a MUX plus control logic, the total capacitance is estimated as:

$$C_{total} \approx (8+8) \times 10 \text{ fF} + 8 \times 15 \text{ fF} + 10 \text{ fF} = 380 \text{ fF} \approx 400 \text{ fF}.$$

Assuming $\alpha = 0.2$, the dynamic power is:

$$P_{dynamic} = 0.2 \times 400 \times 10^{-15} \times (1.2)^2 \times 50 \times 10^6 \approx 5.76 \text{ } \mu W.$$

Thus, the estimated dynamic power consumption is approximately 5–6 microwatts.

I. INTRODUCTION

High-performance multipliers are essential for modern processors, with designs like Wallace Tree and Dadda offering speed at the cost of area and power. Serial multipliers, while slower, achieve lower complexity and energy use. Previous 8×8 and 16×16 designs show a trade-off between speed and efficiency.[2][3]

II. PROPOSED TECHNIQUE

Advantages and Shortcomings of Serial Binary Multipliers: Serial binary multipliers offer significant advantages in terms of area efficiency, low power consumption, and simplified circuit design, making them ideal for embedded systems and low-power processors like RISC architectures. Since operations occur bit by bit, the hardware complexity is greatly reduced compared to parallel multipliers. However, the main shortcomings are longer computation time and lower throughput, as each bit must be processed sequentially over multiple clock cycles. This makes serial multipliers less suitable for high-speed or high-performance computing applications where faster multiplication is critical.[2]

In binary multiplication, the process involves adding shifted versions of the multiplicand based on each bit of the multiplier. If a multiplier bit is '1', the multiplicand is added; otherwise, no addition occurs. Each addition is followed by a right shift, progressively building the final result through a piecewise sum of partial products.

The serial multiplier architecture operates by multiplying the least significant bit (LSB) of the Q register with the entire M register. The result is added to the A register using an adder. After each clock cycle, the contents of A, Q, and the carry bit C are shifted right by one bit. The control unit manages the number of shifts and initiates the operations until all bits of the multiplier have been processed, completing the full 8×8 multiplication in a sequential manner.

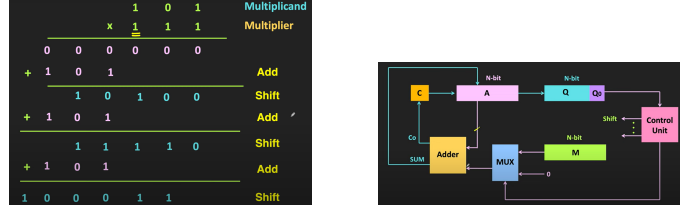


Fig. 1. [1]Left: Step-by-step concept of binary multiplication using the add and shift method. Right: Serial binary multiplier schematic implementing the add and shift approach with A, Q, M, and C registers.

III. BUILDING BLOCKS

A. Basic Gates

First, we designed the basic gates required for building the serial binary multiplier. The fundamental gates implemented at the transistor level include the Inverter, NAND gate, NOR gate, XOR gate, and AND gate. These gates form the backbone of the entire design.

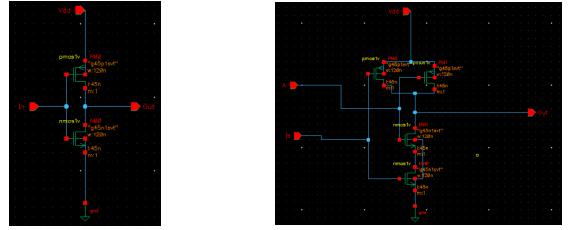


Fig. 2. Left: Inverter gate schematic. Right: NAND gate schematic.

The NOR gate and OR gate were also designed. The OR gate was implemented using a combination of a NOR gate followed by an inverter, based on standard CMOS combinational logic techniques.

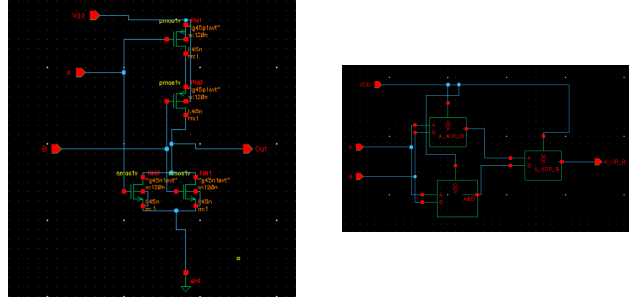


Fig. 3. Left: NOR gate schematic. Right: OR gate schematic.

Finally, XOR and AND gates were designed. The XOR gate is essential for binary addition, while the AND gate was constructed by inverting the output of a NAND gate.

B. Intermediate Circuits

Using the previously designed basic gates, we constructed several intermediate building blocks. These circuits are essential for implementing the serial binary multiplier architecture.

First, 1-bit and 8-bit multiplexers were built for data selection operations.

Then, D-Latch and D Flip-Flop circuits were created to implement storage elements like registers.

Finally, a Carry Adder block was implemented for addition operations within the multiplier.

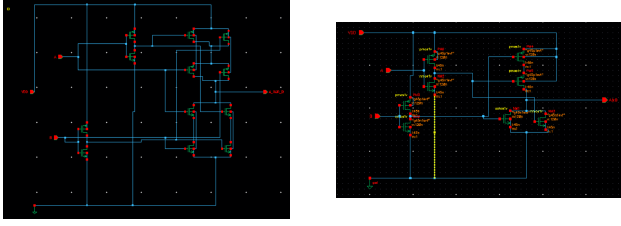


Fig. 4. Left: XOR gate schematic. Right: AND gate schematic.

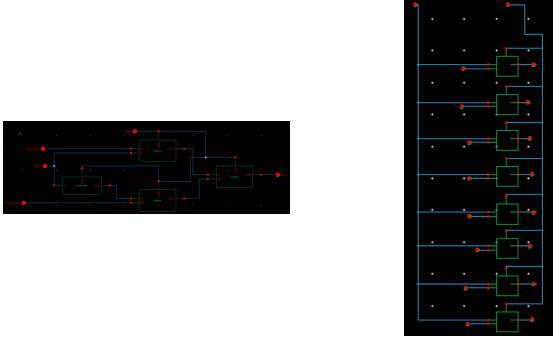
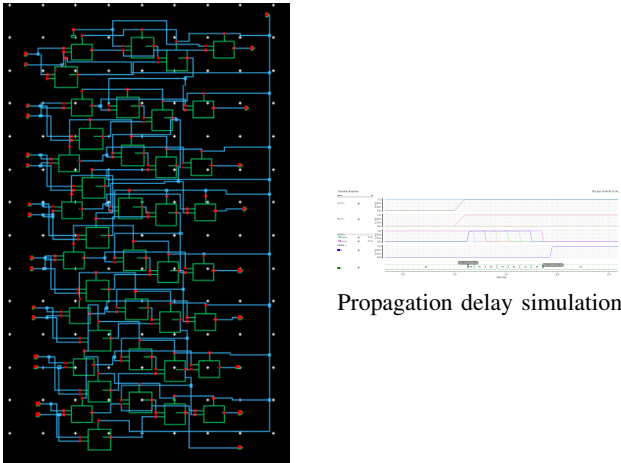


Fig. 5. Left: 1-bit Multiplexer schematic. Right: 8-bit Multiplexer schematic.



Carry Adder schematic

Fig. 7. Left: Carry Adder schematic. Right: Simulation showing the propagation delay of the adder, calculated as $1.85761 \text{ ns} - 1.13182 \text{ ns} = 0.72579 \text{ ns}$.

IV. HIGHER LEVEL COMPONENTS

At a higher level of abstraction, the serial multiplier design includes the Shift Register and the Control Unit. These components are constructed using the intermediate circuits previously discussed.

A. Shift Register (PLSR)

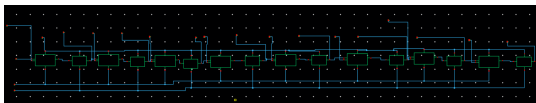


Fig. 8. Parallel Load Shift Register (PLSR) built using D Flip-Flops and 1-bit multiplexers. It allows parallel loading of input values and serial shifting during the multiplication process.

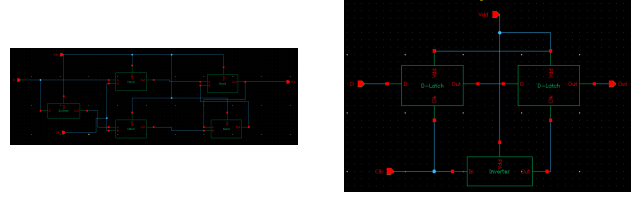


Fig. 6. Left: D Latch schematic. Right: D Flip-Flop schematic.

B. Control Unit Development

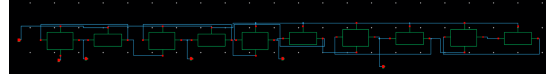


Fig. 9. Counter constructed with D Flip-Flops where each flip-flop clock input is connected through an inverter, implementing binary counting operation.

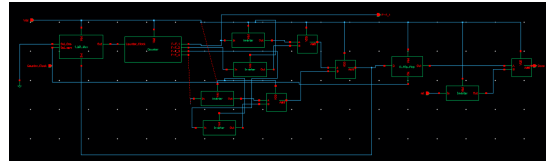


Fig. 10. First version of the Control Unit generating Shift and Add Enable signals using the Counter output.

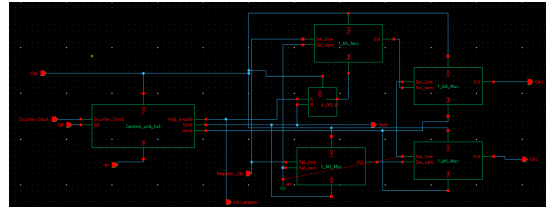


Fig. 11. Improved Control Unit managing clock gating for the shift registers, ensuring clocks are active only during shifting and addition operations.

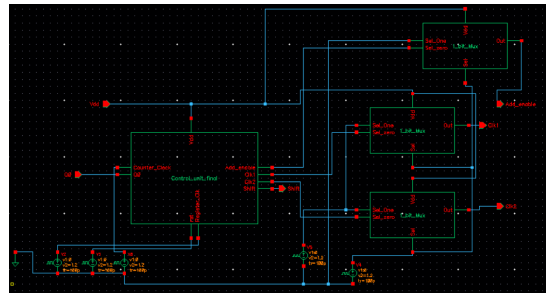


Fig. 12. Final Control Unit integrating Vpulse signals to introduce delays for initialization, allowing safe loading of input values and stable operation at startup.

V. CONTROL UNIT SIGNAL BEHAVIOR

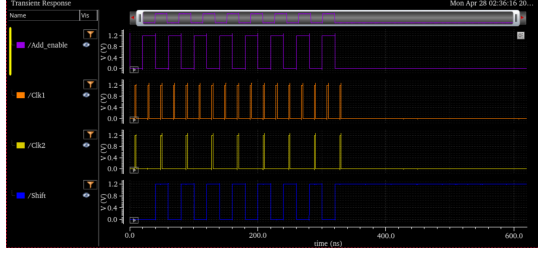


Fig. 13. Simulation of the control unit signals. The Shift Enable signal is active after the initial load period, toggling every half clock cycle. The Add Enable signal is active during the opposite half-cycle when $Q_0 = 1$. The clk2 signal generates a pulse during each shift operation to enable data shifting. The clk1 signal pulses both during shift operations and, when $Q_0 = 1$, during addition cycles. In this simulation, it is assumed that $Q_0 = 1$ for illustration purposes.

A. Final Multiplier Design

By combining all the previously designed components — including the basic gates, intermediate circuits, the Parallel Load Shift Register (PLSR), and the Control Unit — the complete serial binary multiplier was implemented. The system sequentially performs multiplication using the add and shift method, managing operations through controlled clock gating and signal generation.

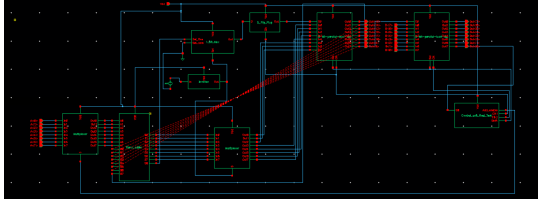


Fig. 14. Complete transistor-level schematic of the 8x8-bit serial binary multiplier combining all major subsystems.

VI. SIMULATION RESULTS

To validate the functionality of the serial binary multiplier, several test cases were simulated. The results confirm that the multiplier produces correct outputs for various input combinations.

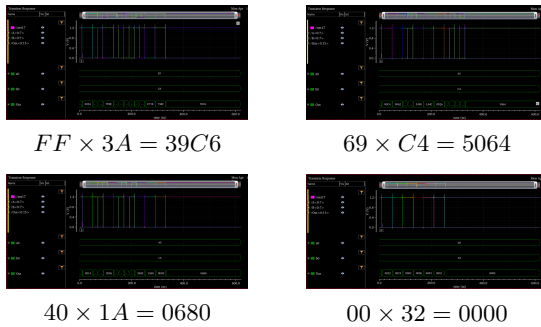
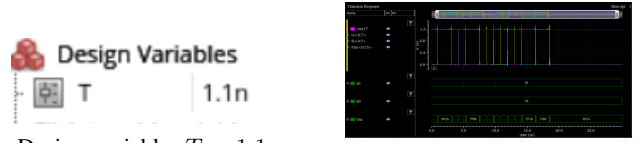


Fig. 15. Simulation results for four different input cases validating the functionality of the serial binary multiplier.

VII. PROPAGATION DELAY ANALYSIS

The propagation delay of the serial binary multiplier was analyzed by reducing the clock period. Testing at different clock periods validates the maximum operating frequency of the circuit.

First, the system was tested at $T = 1.1$ ns. The simulation shows that the multiplier continues to operate correctly at this clock period.



Design variable: $T = 1.1$ ns

Simulation result: Functional at $T = 1.1$ ns

Fig. 16. Validation of correct operation at $T = 1.1$ ns clock period.

Subsequently, when the clock period was reduced to $T = 1.0$ ns, the system failed to complete the multiplication correctly, illustrating timing violations due to insufficient propagation time.



Design variable: $T = 1.0$ ns

Simulation result: Failure at $T = 1.0$ ns

Fig. 17. Illustration of incorrect operation when the clock period is reduced to $T = 1.0$ ns, confirming the timing limitations.

VIII. AVERAGE POWER DISSIPATION

To estimate the average power consumption of the serial binary multiplier, we used the Cadence calculator tool. The simulation was performed with a clock period of $T = 20$ ns. Results were averaged over five experiments to obtain reliable measurements.

The expression used to calculate the average power is shown below.

$$\text{average}(1.2 * (-IT("V0/PLUS")))$$

Fig. 18. Expression used in the Cadence calculator to measure average power dissipation.

The following figures show examples of the output average power values obtained during the experiments.

average(1.2*(-IT("V0/PLUS")))	7.983E-6	6.731E-6	5.144E-6	4.230E-6	9.516E-6
Average Power Experiments 1-2	Average Power Experiments 3-4-5				

Fig. 19. Examples of measured average power dissipation results.

By averaging the five measured values, the estimated average power dissipation of the design was approximately $6.79 \mu\text{W}$, which is slightly larger than the initial estimation, likely due to heavier capacitive loading and/or a higher effective activity factor during operation.

REFERENCES

- [1] All About Electronics, "Serial Binary Multiplier / Sequential Binary Multiplier (using Add and Shift Method)," YouTube, Mar. 20, 2021. Available: https://www.youtube.com/watch?v=5N1FY8EC_8
- [2] GeeksforGeeks, "Sequential Binary Multiplier," *GeeksforGeeks*, Available at: <https://www.geeksforgeeks.org/sequential-binary-multiplier/>
- [3] S. G. Bhat, "Comparative Study of Dadda and Wallace Tree Multiplier Using 180nm Technology," *International Journal of Trend in Research and Development (IJTRD)*, Volume 3, Issue 4, June 2016. Available: <https://www.ijtrd.com/papers/IJTRD3935.pdf>